Code Generation

What about the back half of the compiler?

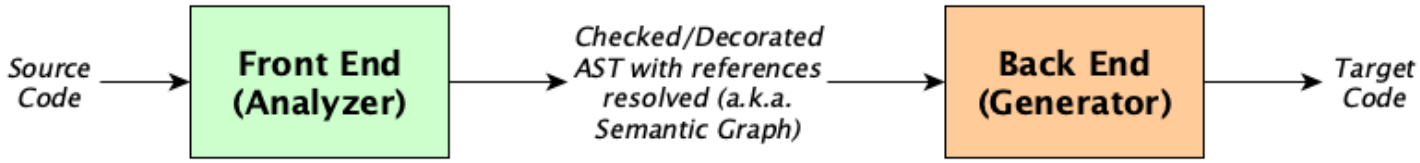
CONTENTS

What is Code Generation? • Goals of Code Generation • What Kind of Code Can We Generate? • Two More Questions • Transpiling • Virtual Machines and Backend Language Libraries • Targeting a Typical Processor • Code Generator Generators • Summary

What is Code Generation?

The first part of a compiler **analyzes** the source code into a structure that carries the meaning of the program; this structure is generally the abstract syntax tree that's been checked and decorated. (Remember decorated means all identifier references have been resolved.)

From this structure we can **generate** the corresponding code in some other language, the target language. This is what a code generator does.



Some compilers generate twice: they first generate code in some "intermediate language" like SIL, LLVM IR, HIR, MIR, CIL, etc. *Then* they do the "real" code generation into a target language that is directly runnable (or really close to it), like virtual machine code, assembly language, or machine language.



Goals of Code Generation

Here are the three goals of any good code generator:

Correctness

It should produce correct code

Efficiency

It should produce efficient code

Self-Efficiency

It should itself be efficient

***Exercise**: The last two goals are seemingly in conflict. Discuss.*

What Kind of Code Can We Generate?

There is no one way to do code generation. There is no one target of a code generator. A code generator can produce:

- **Code in another high-level language**. Don't knock it. If the target high-level language has a good compiler already, this is a great strategy. Stand on the shoulders of giants right?
- **C**. Oh wait, C is a high-level language. Sort of. Wait no. No lambdas. But then you can let the C compiler take over to do the rest.
- **An intermediate representation** like LLVM, that has backends for dozens of architectures.
- **Executable virtual machine code**, such as for the JVM or BEAM. Targeting a virtual machine is a ton of fun. Learning about a VM and its instruction set is a great experience. You'll get a great feeling of accomplishment. And you can run this code, too.
- **Assembly language**: Because it's so fulfilling to learn about machine architecture, especially pipelines, caches, and of course, registers! You need to know your machine. Is it a RISC or CISC processor or some kind of novel architecture like a dataflow processor or somthing with hundreds of cores? What are the

relative instruction speeds? How are its addressing modes? Does it have out-of-order instruction? Any cool idioms?

- **Relocatable machine language**: you've done a lot of assembly that the assembler would have done for you (resolving symbols), but at least you will let the linker do the rest.
- **Absolute machine language**: you've resolved all the memory locations; in other words you spent a lot of time doing what assemblers and linkers would do for you already.

Are there other kinds of code?

Two More Questions

Also consider:

- Should we generate awesome code on the fly? Or let a separate optimizing pass clean up after our generator? Or not optimize at all? For example, naive code generation can lead to not so great code:

```
x = x + 1;        mov rax, x_0     ┐
                  add rax, 1       ├     just need inc x_0
                  mov x_0, rax     ┘

x = y + c;        mov rax, y_5
z = x + 4;        add rax, c_8
                  mov x_2, rax     ; Redundant if x not used again
                  mov rax, x_2     ; Redundant
                  add rax, 4
                  mov z_7, rax
```

- Should we stick in line number information so a debugger can take over if our program is stopped intentionally (or crashes)?

Transpiling

JavaScript makes a great target language? Why? It's got first-class functions and async support, and that Node.js thing runs on that awesome V8. And the ecosystem! How can we target JS?

Start by browsing an existing example. This one, for the language Carlos, translates the decorated AST (Semantic Graph) directly into JavaScript (there's no need to use an IR if you are targeting a high-level language):

generator.js

```
// The code generator exports a single function, generate(program), which
// accepts a program representation and returns the JavaScript translation
// as a string.

import { voidType, standardLibrary } from "./core.js"

export default function generate(program) {
  // When generating code for statements, we'll accumulate the lines of
  // the target code here. When we finish generating, we'll join the lines
  // with newlines and return the result.
  const output = []

  const standardFunctions = new Map([
    [standardLibrary.print, x => `console.log(${x})`],
    [standardLibrary.sin, x => `Math.sin(${x})`],
    [standardLibrary.cos, x => `Math.cos(${x})`],
    [standardLibrary.exp, x => `Math.exp(${x})`],
    [standardLibrary.ln, x => `Math.log(${x})`],
    [standardLibrary.hypot, ([x, y]) => `Math.hypot(${x},${y})`],
    [standardLibrary.bytes, s => `[...Buffer.from(${s}, "utf8")]`],
    [standardLibrary.codepoints, s => `[...(${s})].map(s=>s.codePointAt(0))`],
  ])

  // Variable and function names in JS will be suffixed with _1, _2, _3,
  // etc. This is because "switch", for example, is a legal name in Carlos,
  // but not in JS. So, the Carlos variable "switch" must become something
  // like "switch_1". We handle this by mapping each name to its suffix.
  const targetName = (mapping => {
    return entity => {
      if (!mapping.has(entity)) {
        mapping.set(entity, mapping.size + 1)
      }
      return `${entity.name}_${mapping.get(entity)}`
    }
  })(new Map())

  const gen = node => generators?.[node?.kind]?.(node) ?? node

  const generators = {
    // Key idea: when generating an expression, just return the JS string; when
    // generating a statement, write lines of translated JS to the output array.
    Program(p) {
```

```
  p.statements.forEach(gen)
},
VariableDeclaration(d) {
  // We don't care about const vs. let in the generated code! The analyzer has
  // already checked that we never updated a const, so let is always fine.
  output.push(`let ${gen(d.variable)} = ${gen(d.initializer)};`)
},
TypeDeclaration(d) {
  // The only type declaration in Carlos is the struct! Becomes a JS class.
  output.push(`class ${gen(d.type)} {`)
  output.push(`constructor(${d.type.fields.map(gen).join(",")}) {`)
  for (let field of d.type.fields) {
    output.push(`this[${JSON.stringify(gen(field))}] = ${gen(field)};`)
  }
  output.push("}")
  output.push("}")
},
StructType(t) {
  return targetName(t)
},
Field(f) {
  return targetName(f)
},
FunctionDeclaration(d) {
  output.push(`function ${gen(d.fun)}(${d.params.map(gen).join(", ")}) {`)
  d.body.forEach(gen)
  output.push("}")
},
Variable(v) {
  // Standard library constants just get special treatment
  if (v === standardLibrary.π) return "Math.PI"
  return targetName(v)
},
Function(f) {
  return targetName(f)
},
Increment(s) {
  output.push(`${gen(s.variable)}++;`)
},
Decrement(s) {
  output.push(`${gen(s.variable)}--;`)
},
Assignment(s) {
```

```
  output.push(`${gen(s.target)} = ${gen(s.source)};`)
},
BreakStatement(s) {
  output.push("break;")
},
ReturnStatement(s) {
  output.push(`return ${gen(s.expression)};`)
},
ShortReturnStatement(s) {
  output.push("return;")
},
IfStatement(s) {
  output.push(`if (${gen(s.test)}) {`)
  s.consequent.forEach(gen)
  if (s.alternate?.kind?.endsWith?.("IfStatement")) {
    output.push("} else")
    gen(s.alternate)
  } else {
    output.push("} else {")
    s.alternate.forEach(gen)
    output.push("}")
  }
},
ShortIfStatement(s) {
  output.push(`if (${gen(s.test)}) {`)
  s.consequent.forEach(gen)
  output.push("}")
},
WhileStatement(s) {
  output.push(`while (${gen(s.test)}) {`)
  s.body.forEach(gen)
  output.push("}")
},
RepeatStatement(s) {
  // JS can only repeat n times if you give it a counter variable!
  const i = targetName({ name: "i" })
  output.push(`for (let ${i} = 0; ${i} < ${gen(s.count)}; ${i}++) {`)
  s.body.forEach(gen)
  output.push("}")
},
ForRangeStatement(s) {
  const i = targetName(s.iterator)
  const op = s.op === "..." ? "<=" : "<"
```

```
    output.push(`for (let ${i} = ${gen(s.low)}; ${i} ${op} ${gen(s.high)}; ${i}++) {`)
    s.body.forEach(gen)
    output.push("}")
  },
  ForStatement(s) {
    output.push(`for (let ${gen(s.iterator)} of ${gen(s.collection)}) {`)
    s.body.forEach(gen)
    output.push("}")
  },
  Conditional(e) {
    return `((${gen(e.test)}) ? (${gen(e.consequent)}) : (${gen(e.alternate)}))`
  },
  BinaryExpression(e) {
    const op = { "==": "===", "!=": "!==" }[e.op] ?? e.op
    return `(${gen(e.left)} ${op} ${gen(e.right)})`
  },
  UnaryExpression(e) {
    const operand = gen(e.operand)
    if (e.op === "some") {
      return operand
    } else if (e.op === "#") {
      return `${operand}.length`
    } else if (e.op === "random") {
      return `((a=>a[~~(Math.random()*a.length)])(${operand}))`
    }
    return `${e.op}(${operand})`
  },
  EmptyOptional(e) {
    return "undefined"
  },
  SubscriptExpression(e) {
    return `${gen(e.array)}[${gen(e.index)}]`
  },
  ArrayExpression(e) {
    return `[${e.elements.map(gen).join(",")}]`
  },
  EmptyArray(e) {
    return "[]"
  },
  MemberExpression(e) {
    const object = gen(e.object)
    const field = JSON.stringify(gen(e.field))
    const chain = e.op === "." ? "" : e.op
```

```
      return `(${object}${chain}[${field}])`
    },
    FunctionCall(c) {
      const targetCode = standardFunctions.has(c.callee)
        ? standardFunctions.get(c.callee)(c.args.map(gen))
        : `${gen(c.callee)}(${c.args.map(gen).join(", ")})`
      // Calls in expressions vs in statements are handled differently
      if (c.callee.type.returnType !== voidType) {
        return targetCode
      }
      output.push(`${targetCode};`)
    },
    ConstructorCall(c) {
      return `new ${gen(c.callee)}(${c.args.map(gen).join(", ")})`
    },
  }

  gen(program)
  return output.join("\n")
}
```

What did you notice from browsing these examples? A few tips and tricks, right? Things like:

- You write a `generate` method for each AST node, and each node's `generate` method invokes the generate method of its sub nodes. The top-level function `gen` **dispatches** to the right generate method
- When writing out high-level code, it is good to over-parenthesize: you should probably not assume your associativities and precedences and arities and fixities are the same as those of your target language.
- We should decorate the ids in the target language with uniquely identifying suffixes, because (1) the set of reserved words may differ between source and target languages, and (2) you never know whether scope rules are the same!
- While not shown in the examples above, character sets may very well be different between the two languages; keep that in mind.
- Inline some standard functions here. (But where standard functions cannot be inlined, your code generator can always write out the function implementation in the target language.)
- If your source language has static type checking, static access checks, static matching checks, static read-only checks, safe optionals, etc., these have already been done in the analyzer, and do not need to be handled in the generator! The Carlos generator above is an example of this.
- However, if your source language is dynamic, then you must generate code to do the type, access, read-only, null, and matching checks!

- Even for very static languages, some checks will still have to be done in generated code, like array length checks (when the array is a variable whose size is unknown until run-time)
- There may be a ton of ad-hoc things that pop up, but these are fun challenges.

Here is a view of some of the Carlos functions in table form:

Graph Fragment	JavaScript
program / \ s_1 .. s_n	s_1'; . . s_n';
vardecl / | \ v T E	let v' = E'

structtypedecl
/ \
t structtype
/ \
field .. field
/ \ / \
x_1 T_1 x_n T_n
	class t' {
	constructor (x_1', .., x_n') {
	this.x_1' = x_1';
	...
	this.x_n' = x_n';
	}
	}

functiondecl
/ | \
function T B
/ | \
f param .. param
/ \ / \
x_1 T_1 x_n T_n
	function f(x_1', .., x_n') {
	B
	}

inc | V	V'++
dec | V	V'--
assign / \ V E	V' = E'
break	break
return	return

return | E	return E'
if / \ E B	if (E') { B' }
if / | \ E B_1 B_2	if (E') { B_1' } else { B_2'} // But no braces around B_2 if it's an if-statement
while / \ E B	while (E') { B' }
repeat / \ E B	for (let i = 0; i < E'; i++) { B' } // (note: i is a new variable)
for-upto / | | \ x E_1 E_2 B	for (let x' = E_1'; x' < E_2'; x'++) { B' }
for-through / | | \ x E_1 E_2 B	for (let x' = E_1'; x' <= E_2'; x'++) { B' }
for-collection / | \ x A B	for (let x' of A') { B' }
conditional / | \ E E_1 E_2	((E') ? (E_1') : (E_2'))

+
/ \
E_1 E_2
	(E_1' + E_2)
	// same for all other binary ops
	// translated where necessary (e.g. == → ===)

-
|
E
	(- E')
	// same for all other unary ops
	// except "some", which is a no-op in JS

emptyoptional	undefined
subscript / \ A E	A'[E']
arrayexpression / \ E_1 ..... E_n	[E_1', ..., E_n']

member / \ O F	O'["F'"] // quotes are important
fn-call / | \ F E_1 .. E_n	F'(E_1', ..., E_n') // Standard Library functions handled inline
ctor-call / | \ C E_1 .. E_n	new C'(E_1', ..., E_n')

Virtual Machines and Backend Language Libraries

LLVM

LLVM is, well, it's huge, it's impressive, and lots of compilers for modern languages are using it. It is a library with a ton of stuff, including an intermediate language and backends. It's official name is **The LLVM Compiler Infrastructure Project**.

A take on modern compiler construction from Andi McClure:

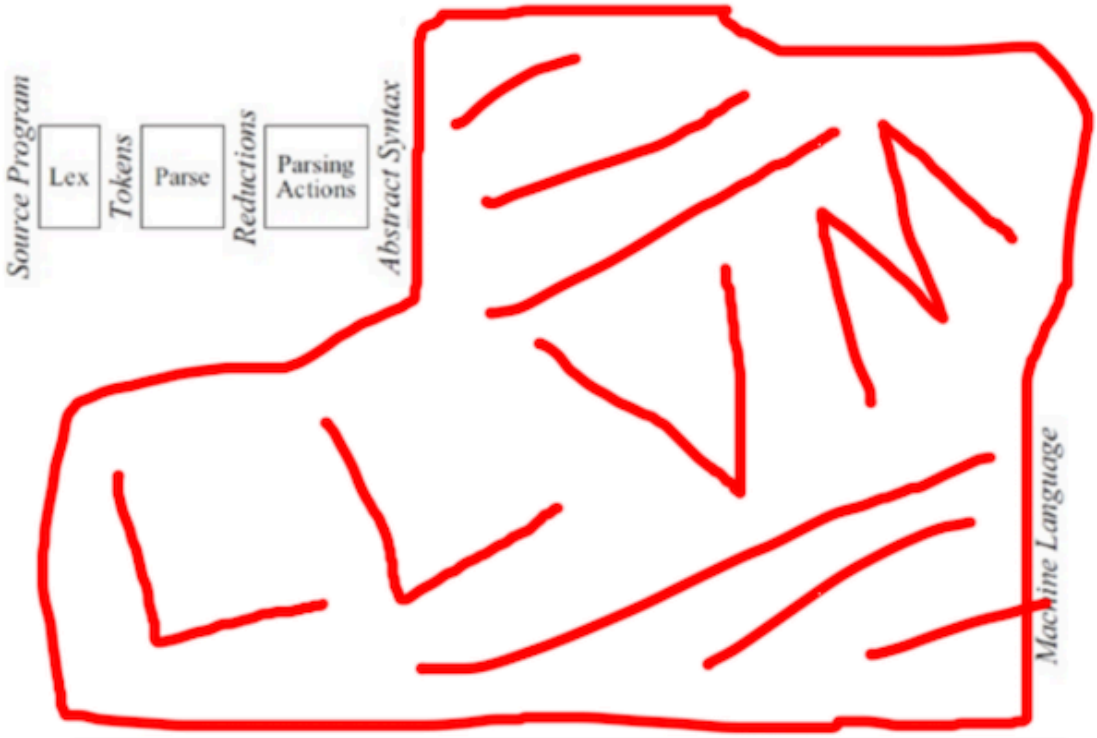


FIGURE 1.1. Phases of a compiler, and interfaces between them

***Exercise**: Read the whole blog post from whence came that awesome picture.*

JVM

Read about the JVM at Wikipedia and about its formal specification.

Web Assembly

Read about Web Assembly

***Exercise**: Explain WASM, briefly, in your own words.*

Write your own VM!

SERIOUSLY. DO IT.

Targeting a Typical Processor

Okay let's say you're going to bypass LLVM or anything else and you are going straight to assembly. Or, let's say you are joining the LLVM team and want to enhance the project. Or you want to write a new backend for an intermediate representation. Lots of good stuff to know. Here are some notes that just scratch the surface of generating assembly language.

Address Assignment

TODO - Fortran no recursion local vars, even return val on stack

TODO - C, show assembly for recursive function

TODO - C, show space for local vars

TODO - motivate stack frames

TODO - motivate dynamic links

TODO - Three approaches to static scope

TODO - Static link

TODO - Single display

TODO - Display in every frame

TODO - No need for display or static link in leaf subroutine

TODO - Who saves the registers

TODO - Inlining?

TODO - Summary of questions and concerns

Instruction Selection

TODO

Register Allocation

TODO

Code Generator Generators

TODO

Summary

We've covered:

- ✅ -
- ✅ -

Code Improvement

People like to write efficient and compact code. Let's try to get a compiler to write this kind of code, too.

CONTENTS

Motivation and Goals • Where Can Optimization Be Done? • Optimization Strategies • A Catalog of Optimization Techniques • Links • Examples • Further Reading • Summary

Motivation and Goals

We want a compiler to produce *good* code, at least we usually do. We may wish to "optimize" one or more of:

- Program size
- Program speed
- Resource usage (page swaps, caches, file handles, threads, etc.)

General **optimization** is undecidable, and many specific optimization tasks happen to take provably exponential time or are NP-Hard. So we normally speak of code **improvement**.

The general rule is to do those things that give the **most** improvement for the **least** amount of work, so for instance, you might see compilers that don't bother doing any improvement outside loops. Still, there are different degrees of optimization aggressiveness:

- No improvements at all
- Only few basic simplifications on AST nodes or tuples (peephole optimization)
- Attempt near-optimal code **within basic blocks** (by removing redundancies like common subexpressions, doing smart instruction scheduling and register allocation, etc.)
- **Intraprocedural optimization** (improving across basic blocks, but still within a function, e.g., loop optimization using data flow analysis)
- **Interprocedural optimization**

Ideally, optimizations should satisfy two criteria:

SAFETY

The optimized code does the “same thing” as before

PROFITABILITY

The optimized code is actually smaller or faster

but as we’ll see, unsafe and unprofitable optimizations are sometimes done in practice.

Where Can Optimization Be Done?

There is no single optimization phase in a compiler. Lots of things happen to improve code:

1. The programmer needs to find a better algorithm. Not many (if any) compilers can turn bubblesort into quicksort.
2. The intermediate code generator can be smart about how it generates intermediate code.
3. The compiler can make a pass or two or three over the intermediate code to improve it.
4. The code generator can be smart, with clever instruction selection, by employing register tracking, etc.
5. The compiler can make a pass or two or three over the target code to improve it.
6. The run time system can adapt to the way a program is running, possibly recompiling and relinking on the fly.
7. The programmer profiles the running code and finds out where the memory leaks and bottlenecks and excessive resource consumption are occurring, and rewrites the code accordingly.

Optimization Strategies

In general, you’ll try to:

- Use cheaper, more efficient operations
- Reuse results rather than recompute them

- Maximize the use of registers, caches, and memory
- Reduce the amount of code (e.g., remove dead and useless code)
- Reduce conditional jumps (they make branch prediction difficult)
- Optimize the use of pipelines and cores

But note the need for tradeoffs! You can get rid of some jumps by unrolling loops, but this makes more code.

> ***Exercise**: What are other conflicts?*

A Catalog of Optimization Techniques

Machine Independent Optimizations

- Constant Folding
- Algebraic Simplification (e.g. operand reordering)
- Strength Reduction
- Unreachable Code Elimination
- Dead Code Elimination
- Copy Propagation
- Common Subexpression Elimination
- Loop Unrolling
- Loop Fission
- Loop Fusion
- Loop Inversion
- Loop Interchange
- Loop Invariant Factoring
- Jump Threading
- Induction Variable Simplification
- Tail Recursion Elimination
- Inline Expansion

Mostly Machine Independent Optimizations

- Efficient address computation
- Simplified stack frames
- Static allocation of frames
- Inlining

Machine Dependent Optimizations

- Careful instruction selection, using an understanding of caches, pipelines, branch prediction algorithms, scheduling rules, alignment requirements
- Smart register allocation
- Smart use of address modes
- Balancing across multiple CPUs

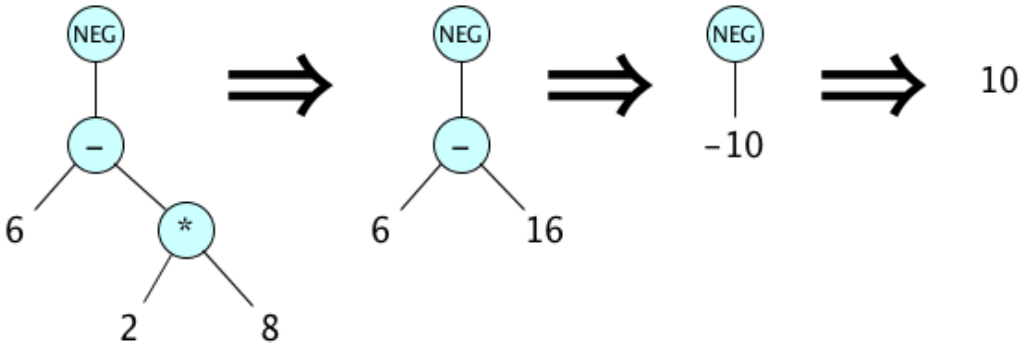
Links

- Compiler Optimization at Wikipedia
- Wikipedia's Compiler Optimizations Category Page
- Agner Fog's Software Optimization Resources
- A pretty good reference

Examples

Constant Folding

Why wait until the target program is run to do simple computations? Let the compiler do it. Example:



Remember we can always write trees as strings, so we'll do that from now on:

`(NEG (– 6 (* 2 8)))`	`(NEG (– 6 16))`	`(NEG –10)`	`10`

Strength Reduction

You can replace expensive operations with cheaper ones:

`(+ x 0)`	`x`
`(- x 0)`	`x`
`(- x x)`	`0`
`(- 0 x)`	`(NEG x)`
`(+ x (NEG y))`	`(- x y)`
`(- x (NEG y))`	`(+ x y)`
`(* x 0)`	`0`
`(* x 1)`	`x`
`(/ x 1)`	`x`
`(/ x x)`	`1`
`(% x 1)`	`0`
`(% x x)`	`0`
`(/ 0 x)`	`0`
`(% 0 x)`	`0`
`(< x x)`	`false  -- also for > and NOTEQ`
`(EQ x x)`	`true  -- also for <= and >=`
`(AND x false)`	`false`
`(OR x true)`	`true`
`(NOT <= x y)`	`(> x y)`
`(>= (ABS x) 0)`	`true`
`(* x 4)`	`(SHL x 2)`
`(/ x 4)`	`(SHR x 2)  -- arithmetic right shift`

***Exercise**: Think up at least a dozen more of these.*

You can even "invent" some powerful operations:

`(+ (* x 90) y)`	`(MULADD x 90 y)`
`(AND (<= x 3) (>= x -10))`	`(INRANGE x -10 3)`

> *By "inventing" powerful operations, we mean creating new AST nodes representing operations that may not appear in your source language. That is, a language may not have INC, MULADD, INRANGE or a host of other operators, but during optimization there is nothing wrong with adding these new nodes! In fact, some can be created for the sole purpose of allowing better code to be generated.*

Operand Reordering

Use commutativity, associativity, and distributivity to either reduce the number of operations, or do things like "put the more complex code on the left:"

`(+ x (- y z))`	`(+ (- y z) x)`	`-- commutativity`
`(* x (* y z))`	`(* (* y z) x)`	`-- associativity`
`(- (* x 5) (* y 5))`	`(* (- x y) 5)`	`-- distributivity`

Complex on the left simplifies register allocation on some machines.

But watch out: associativity and distributivity changes may not preserve overflow semantics.

```
$ swift
  1> let x: Int8 = 70
x: Int8 = 70
  2> let y: Int8 = 80
y: Int8 = 80
  3> let z: Int8 = 30
z: Int8 = 30
  5> (x + (y - z))
$R0: Int8 = 120
  4> (x + y) - z
Execution interrupted. Enter code to recover and continue.
Enter LLDB commands to investigate (type :help for assistance.)
```

Assignment Simplification

As long as assignment doesn't have hidden side-effects, you can do things like this:

`(ASSIGN x x)`	`(NOP)`
`(ASSIGN x (+ x 1))`	`(INC x)`
`(ASSIGN x (- x 1))`	`(DEC x)`
`(ASSIGN x (+ x y))`	`(+= x y)`

> *Beware of languages that allow for side effects or triggers to take place on assignment. For example Ada allows assignment triggers, and C++ allows you to overload the assignment operator. The optimzations above prevent those effects and triggers, hence those optimizations are strictly speaking unsafe.*

Unreachable Code Elimination

If you can't ever get to some code, don't generate it. With trees:

`(IF true T1 T2)`	`T1`

`(IF false T1 T2)`	`T2`
`(BLOCK T1 T2 RETURN T3 T4)`	`(BLOCK T1 T2 RETURN)`
`(WHILE false T)`	`(NOP)`
`(FOR i emptyrange T)`	`(NOP)`

Other general strategies:

- Remove all tuples after an unconditional control transfer (JUMP, RETURN, THROW), up to, but not including, the next LABEL.
- Remove all basic blocks with no predecessors
- Remove all private functions or methods that are never called

Dead Code Elimination

If a local variable or by-value parameter is assigned to but never subsequently used, you can delete the assignment. As usual, do this only if no implicit operations have side effects, and watch out for aliasing.

```
void f() {
    int p = 5;
    int* q = &p;
    p = 3;          // LOOKS LIKE DEAD CODE BUT IT IS NOT!!
    cout << *q;
}
```

Copy Propagation

After an assignment `x := y` (i.e., `(COPY y x)`), replace all *uses* of `x` with `y` until such point that the values of `x` or `y` could be different. This might turn the copy into dead code which you can eliminate. Given the source code:

```
w := 4;
a = w - 7 + (z * w)
```

then optimizing the AST proceeds as follows:

`(BLOCK`
`  (ASSIGN w 4)`
`  (ASSIGN a`
`  (+ (- w 7) (* z w))`
`)`
	`(ASSIGN a`
	`  (+ (- 4 7) (* z 4)))`
		`(ASSIGN a`
		`  (+ (- 3) (* z 4))`
			`(ASSIGN a`
			`  (- (* z 4) 3))`

And with Tuples:

(COPY,4,w) (MUL,z,w,t1) (SUB,w,7,t2) (ADD,t2,t1,a)	(MUL,z,4,t1) (SUB,4,7,t2) (ADD,t2,t1,a)	(MUL,z,4,t1) (COPY,-3,t2) (ADD,t2,t1,a)	(MUL,z,4,t1) (ADD,-3,t1,a)	(MUL,z,4,t1) (SUB,t,3,a)

Common Subexpression Elimination

If an expression appears multiple times, **AND ITS VALUE CANNOT CHANGE IN BETWEEN THESE EVALUATIONS**, then you can just evaluate it once:

`x * 3 + x * 3 + 10`	`_t = x * 3;` `_t + _t + 10;`	
`a[i][j] = a[i][j] + 1`	`_t = &a[i][j];` `(*_t)++;`	

`x = a[i];`
`a[i] = a[j];`
`a[j] = x;`
	`_t0 = &a[i];`
	`x = *_t0;`
	`_t1 = &a[j];`
	`*_t0 = *_t1;`
	`*_t1 = x;`
		`--After CP and DCE`
		`_t0 = &a[i];`
		`_t1 = &a[j];`
		`*_t0 = *_t1;`
		`*_t1 = *_t0;`

Expressions that look common may not be, due to pointers, aliasing, references, arrays, etc.

```
x = a[i] - 85;
a[j] = 21;          // a[i] is NOT common
print(a[i]);        // ... because maybe i == j
```

If your language has references, be very careful

```
x = j * 8 - y;
q = 10;             // Make sure you never had done
c = j * 8 - y       // ... something like int& q = j
```

Conditional Jump Tuple Compression

A naive translation into tuples for:

```
if (x < 3) { ... }
```

can be optimized like so:

(LT, x, 3, t0) (JZ, t0, L3)	(JGE, x, 3, L3)

This can be very useful since the naive translation of the first tuple sequence to x86 assembly would be:

```
cmp x, 3
setl dl
movzx eax, dl
test eax, eax
jz L3
```

and for the second:

```
cmp x, 3
jg3 L3
```

Loop Unrolling

Loops can have a performance issue, since time is required to execute those jumps, and conditional jumps can cause slowdowns because of the potential for branch misprediction.

`for i in 1..4 {`
`  T`
`}`
	`i = 1; T;`
	`i = 2; T;`
	`i = 3; T;`
	`i = 4; T;`

`(FOR i (RANGE 1 4) T)`	`(BLOCK`
	`  (ASSIGN I 1) T`
	`  (ASSIGN I 2) T`
	`  (ASSIGN I 3) T`
	`  (ASSIGN I 4) T`
	`)`

Loop unrolling is commonly followed by copy propagation!

`for i in 1...4 {`
`  print(i * 2)`
`}`
	`i = 1; print(i * 2);`
	`i = 2; print(i * 2);`
	`i = 3; print(i * 2);`
	`i = 4; print(i * 2);`
		`print(1 * 2);`
		`print(2 * 2);`
		`print(3 * 2);`
		`print(4 * 2);`
			`print(2);`
			`print(4);`
			`print(6);`
			`print(8);`

Look unrolling might conflict with cache issues, since the code is getting bigger.

Note: you can *partially unroll*, that works too.

Loop Fusion

Suppose you know that you had an array of structs:

```
struct {
  int a;
  int b;
} s[5000];
```

then two loops accessing the a's and b's independently could be **fused** into one to improve locality (and avoid destroying performance due to cache unloading and refilling:

`for i in 0..<n {`
`  ... s.a[i] ...`
`}`
`for i in 0..<n {`
`  ... s.b[i] ...`
`}`
	`for i in 0..<n {`
	`  ... s.a[i] ...`
	`  ... s.b[i] ...`
	`}`

Loop Fission

Suppose you know that you had two parallel arrays in a struct:

```
struct {
  int a[5000];
  int b[5000];
} s;
```

then one loop that iterated both array could be **split** into two loops accessing the a's and b's independently to improve locality (and avoid destroying performance due to cache unloading and refilling:

`for i in 0..<n {`
`  ... s.a[i] ...`
`  ... s.b[i] ...`
`}`
	`for i in 0..<n {`
	`  ... s.a[i] ...`
	`}`
	`for i in 0..<n {`
	`  ... s.b[i] ...`
	`}`

Loop Inversion

This interesting technique replaces a while loop with a do-while loop in an if statement, looking like this at the source code view (BUT DON'T REALLY DO THIS IN YOUR SOURCE CODE, LET THE COMPILER DO IT):

`while (condition) {`
`  body`
`}`
	`if (condition) {`
	`  do {`
	`  body`

```
  } while (condition);
}
```

This tends to generate better machine code, because there are fewer jumps. Here it is in tuples:

```
top:
  condition
  (JZ, bottom)
  body
  (JUMP top)
bottom:
  ...
```

```
  condition
  (JZ, bottom)
top:
  body
  condition
  (JNZ, top)
bottom:
  ...
```

The last iteration of the loop has two jumps not taken in the optimized version compared to the unoptimized version. Pretty useful if this is a nested loop!

For a loop executed n times	Unoptimized Version			Optimized Version	
	Conditional Jumps		Unconditional Jumps Taken	Conditional Jumps	
	Taken	Fallen Through		Taken	Fallen Through
0	1	0	0	1	0
1	1	1	1	0	2
2	1	2	2	1	2
3	1	3	3	2	2
4	1	4	4	3	2
5	1	5	5	4	2
100	1	100	100	99	2

> ***Remember these are *compiler* optimizations***
>
> *Don't use the examples on this page as an example of how you, as a programmer, should be writing source code. The page discusses optimizations that a compiler can do for you. You (the programmer) are supposed to write clear, readable code. Let the compiler optimize. It's generally much better at that you are.*

Loop Interchange

If you know that your matrices are laid out in row-major (as in C) or column-major (as in Fortran) order, and you detect that a matrix is being processed with nested loops in a manner that is opposite from the layout, and you can tell that the body of the loop is not modifying the matrix in any way dependent on the traversal order, you can switch the traversal order. Here is an example of switching from row order to column order:

```
for i in 0..<m {
    for j in 0..<n {
        ... a[i, j] ...
    }
}
```

```
for j in 0..<n {
  for i in 0..<m {
      ... a[i, j] ...
  }
}
```

Loop Invariant Factoring

A **loop invariant** is an expression that is always the same for every execution of the loop. You might consider moving the evaluation of this expression just outside (just before) the loop, so that it evaluated only once. Example:

```
for i in 0..<n {
    for j in 0..<n {
        for k in 0..<n {
            a[i][j][k] = i * j * k
        }
    }
}
```

```
for i in 0..<n {
    _t0 = a[i]
    for j in 0..<n {
        _t1 = _t0[j]
        _t2 = i * j
        for k in 0..<n {
            _t1[k] = _t2 * k
        }
    }
}
```

If t = 100, for example then:

- Without factoring: 3,000,000 array accesses and 2,000,000 multiplications
- With factoring: 1,010,100 array accesses and 1,010,000 multiplications

> ***Factoring may be unsafe***
>
> *You may factor an expression that throws an exception (or crashes), and if it does the exception would be thrown at the wrong place, perhaps before a side-effect which could completely change the program meaning.*
>
> *A good compiler will only factor expressions it knows cannot throw.*

> ***Factoring may be unprofitable as well as unsafe***

Factoring an invariant from a loop that is executed zero times causes unnecessary and potentially harmful execution of code that should not ever have been run.

Jump Threading

Sometimes a jump will land on a jump (or a throw or a return). Examples (for tuples):

`(JGE, x, 3, L5)`
`some code`
`L5:`
`(JUMP, L7)`
	`(JGE, x, 3, L7)`
	`some code`
	`L5:`
	`(JUMP, L7)`

`(JGE, x, 3, L5)`
`some code`
`L5:`
`(JGE, x, 2, L7)`
	`(JGE, x, 3, L7)`
	`some code`
	`L5:`
	`(JGE, x, 2, L7)`

Here's a nice Jump Threading article.

The basic idea of jump threading is to avoid conditional jumps that you know will always be taken or never be taken, and make sure they are never executed. The same theme appears in transformations such as these (at the AST level):

`(BLOCK` `(IF c T1) (IF c T2))`	`(IF c (BLOCK T1 T2))`
`(BLOCK` `(IF c T1) (IF (NOT c) T2))`	`(IF c T1 T2)`

Again, only perform these if you can prove execution of T1 will not affect c.

Induction Variable Simplification

An **induction variable** is any variable whose value is incremented by a constant each pass through the loop.

An **induction expression** has the form `i*c1+c2` for induction variable `i` and invariants `c1` and `c2`. You can get rid of the multiplication that it is the loop, replacing it with a much cheaper addition! Here's an example:

`for i in 1...n {`
`a[i] = 80 * i + q`
`}`
	`_t = 80 + q`
	`for i in 1...n {`
	`a[i] = _t`
	`_t = _t + 80`
	`}`

In general, replace induction expression `i*c1+c2` by initializing a new variable to `i0*c1+c2` and incrementing by `s*c1` where `s` is the step size

> ***Exercise:*** *But is this okay with floating point numbers that require high precision?*

Tail Recursion Elimination

A function that returns a call to itself can be transformed into a loop. The classic gcd example is:

```
function gcd(x, y) {
  if (y === 0) {
    return x;
  }
  return gcd(y, x % y);
}
```

```
function gcd(x, y) {
  while (true) {
    if (y === 0) {
      return x;
    }
    [x, y] = [y, x % y];
    continue;
  }
}
```

The `continue` is emitted just in case there is more code after the return, but of course if the optimizer can tell there is no code there, then it is unnecessary. Be careful that the `return` is not inside a nested loop!

With tuples, the transformation is like:

```
(JNZ, y, L2)
(RET, x)
(LABEL, L2)
(STARTCALL, gcd)
(PARAM, y)
(MOD, x, y, t0)
(PARAM, t0)
(CALL, gcd, t1)
(RET, t1)
```

```
(LABEL, L1)          -- add this new label at the top
(JNZ, y, L2)
(RET, x)
(LABEL, L2)
(COPY, y, t0)        -- compute new first param
(MOD, x, y, t1)      -- compute new second param
(COPY, t0, x)        -- load new first param
(COPY, t1, y)        -- load new second param
(JUMP, L1)           -- "call" by jumping to top
```

For fun, here are the tail recursion elimination applications of Factorial (remember a should be primed with 1 here):

```
function fact(n, a) {
  if (n === 0) {
    return a;
  }
  return fact(n - 1, a * n);
}
```

```
function fact(n, a) {
  while (true) {
    if (n === 0) {
      return a;
    }
    [n, a] = [n - 1, a * n];
    continue;
  }
}
```

and Fibonacci (remember a and b should be primed with 0 and 1):

```
function fib(n, a, b) {
  if (n === 0) {
    return a;
  }
  return fib(n - 1, b, a + b)
}
```

```
function fib(n, a, b) {
  while (true) {
    if (n === 0) {
      return a;
    }
    [n, a, b] = [n - 1, b, a + b];
    continue;
  }
}
```

Inline Expansion

Inlining saves the overhead of a function call at runtime. In practice it often exposes many other optimizations. Here's a trivial example to show the basic idea, where we **inline** the body of `g` into the point in `f` where it is called. (If this is the only call to `g`, or we inline all calls to `g`, then we don't have to generate a separate function for it at all):

```
function g(x, y) {
  let z = x * y;
  console.log(z / 5);
  return z + 1;
}
function f(x) {
  if (x > 0) {
    console.log(g(10, x));
  }
  return 1;
}
```

```
function f(x) {
  if (x > 0) {
    let __g_x = 10;
    let __g_y = x;
    let __g_z = __g_x * __g_y;
    console.log(__g_z / 5);
    let __g_returnValue = __g_z + 1;
    console.log(__g__returnValue);
  }
  return 1;
```

You should inline when the function is:

- Called only once in the program text
- Extremely small. Obvious candidates are those ridiculous getter methods like `int getX() { return x; }` in Java and C++.

You should strongly consider inlining when the function is:

- Frequently called at runtime (i.e., called in a loop)

You probably don't want to inline

- Recursive functions, but you can partially expand them (kind of like partial loop unrolling)

***Exercise**: Show how a recursive gcd function would be partially inlined. Do three "inlines."*

Further Reading

Nothing wrong with starting at the Wikipedia page for compiler optimizations and clicking from technique to technique and getting lost and absorbed in this incredibly cool subdiscipline of compilers.

Thinking about these topics is good. It might help you see computation in a new way.

Also, read whatever you can about Fran Allen, the 2006 Turing Award winner who did so much of the seminal work in optimizing compilers.

Summary

We've covered:

- ✅ Motivation
- ✅ Where optimizations happen in a compiler
- ✅ What we try to optimize
- ✅ A whole bunch of techniques
- ✅ Where to learn more

Runtime Systems

What do we need to know, and what do we need, when our programs are running?

CONTENTS

Runtime Memory Models • Static Storage • Stack Storage • Heap Storage • Garbaage Collection • Process Management • Thread Management • Summary

Runtime Memory Models

TODO

Static Storage

TODO

Stack Storage

Each invocation of a subroutine has an **activation record**, or **frame** associated with it at runtime, holding some or all of the following:

- Arguments (parameters)
- A link to the caller's stack frame (the *dynamic link*)

- In a language with nested subroutines, a mechanism for accessing non-locals, which could be one of:
  - A static link, or
  - A pointer to the most recent frame at this level, or
  - The whole display
- The return address
- Local variables
- Copies of callee-save registers, if needed
- Temporaries
- Spilled registers

Note that some of the stack frame may be in registers and some of it may be in memory.

Activation records are also called **stack frames**. When the runtime supports closures or concurrency, something like a **spaghetti stack** is required.

Calling Conventions

Calling conventions specify:

- Order of evaluation of arguments (left-to-right, right-to-left, in parallel, unspecified...)
- Where the arguments go (run-time stack, registers, ...)
- Which registers are allowed for arguments, which registers are to be saved by the caller and which by the callee
- Whether the caller or callee has to clean up the stack

Example: C on the Intel64

TODO

Historical Example: C on the IA-32

The C-calling convention on the IA-32 is:

- Arguments are evaluated right to left and pushed on the stack
- Callee must save registers ebx, ebp, esi, edi if it uses them
- Caller must save every other register that it wants preserved
- Return value goes in eax, edx:eax, or st[0]

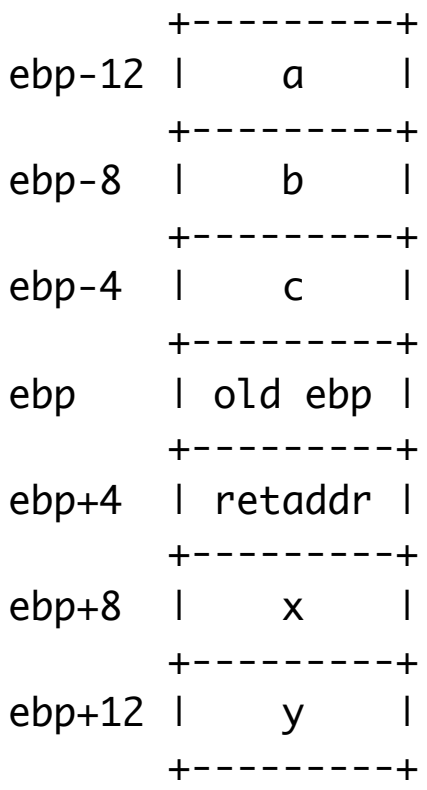
- Register ebp is used to point to the previous stack frame, so the old value of ebp is (almost) always saved.
- Space for local variables in the callee is allocated above the saved ebp.

```
int example(int x, int y) {
  int a, b, c;
  ...
  f(y, a, b, b, x);
  ...
}
```

The prolog/epilog for this function would look like:

```
    push    ebp          ; must save old ebp
    mov ebp, esp         ; point ebp to this frame
    sub esp, ___         ; make space for locals
    ...
    mov esp, ebp         ; clean up locals
    pop ebp           ; restore old ebp
    ret
```

with stack frame:



Example: SPARC

The SPARC uses **register windows**. You can read about these at Wikipedia. Let's start with an example:

```
! ----------------------------------------------------------------------
! writeint
!
! A small program that contains a function to write an unsigned integer value
! to standard output.
! ----------------------------------------------------------------------

        .global _start

        .text

! writeint(unsigned int n) writes n to standard output as text. It is
! inefficient since it is recursive and invokes a system call to write
! each digit.
!
! It is equivalent to the C function
!
!   void writeint(unsigned n) {
!       if (n >= 10) writeint(n / 10);
!       putchar(n + '0');
!   }

writeint:
    save %sp, -96, %sp

    mov 10, %l0                 ! Keep 10 in %l0
    cmp %i0, %l0                ! Is it less than 10?
    bl  writedigit              ! If so just write the digit
    nop
    mov %g0, %y
    udiv %i0, %l0, %l1          ! n / 10 -> %l1
    umul %l1, %l0, %l2          ! n * (n/10) -> %l2
    sub %i0, %l2, %l2           ! n mod 10 -> %l2

    call writeint
    mov %l1, %o0                ! call writeint(n / 10)

    mov %l2, %i0                ! Put digit into %i0 before writing

writedigit:                     ! Writes the digit in %i0
```

```
    add 48, %i0, %l1            ! Character value of the digit
    set digit, %o1
    stb %l1, [%o1]              ! Must be in memory to write
    mov 4, %g1                  ! System call write
    mov 1, %o0                  ! stdout
    mov 1, %o2                  ! number of characters to write
    ta  0

    ret
    restore

! A starting point for the program that just makes a couple of calls
! to the writeint function.

_start:
    call    writeint        ! writeint(5)
    mov 5, %o0

    call    writeint        ! writeint(1312)
    mov 1312, %o0

    sethi   %hi(7308841), %o0   ! writeint(7308841)
    call    writeint
    or  %o0, %lo(7308841), %o0

    mov 1, %g1
    ta  0

    .data
    .align  4
digit:  .skip   4
```

TODO - describe this example

Leaf Subroutines

A leaf subroutine makes no calls. A good compiler can generate code to make a leaf subroutine's activation use the stack frame of its caller. The SPARC has direct support for this. Example:

```
! write_stars(n) writes n stars followed by a newline

write_stars:
    mov %o0, %l0        ! save the argument in %l0
```

```
    mov 4, %g1          ! syscall write
    mov 1, %o0          ! stdout
    set star, %o1       ! message to write
    mov 1, %o2          ! one character in message
write_star:
    tst %l0             ! no more characters to write
    be  write_newline   ! write terminating newline if so
    nop
    ta  0
    ba  write_star      ! go see if more to write
    dec %l0             ! count off one more char as written
write_newline:
    set newline, %o1
    retl
    ta  0               ! write the newline (before returning)
star:
    .ascii  "*"
newline:
    .ascii  "\n"
```

TODO - describe this example

Inlining

A compiler inlines a subroutine call when it replaces the call with the body of the called subroutine (with all argument-parameter bindings resolved, of course). If all calls are inlined, there is no need to generate separate code for the subroutine. Inlining should be faster in general, since there is no call/return overhead (which is substantial for short subroutines). But if there are thousands of such calls you can end up with more compiled code.

And compilers have to be careful when inlining recursive subroutines.

Non Local Lexical Variables

Given:

```
function f(a, b) {
    var x = 1;

    function g(a, y) {
        var z = 3;
```

```
        function h(z) {
            print x;
        }

        function k(c) {
            var x = c;
            if (x < 5) k(c+1); else h(1);
        }

        if (a == 0) g(1, 0); else k(2);
    }

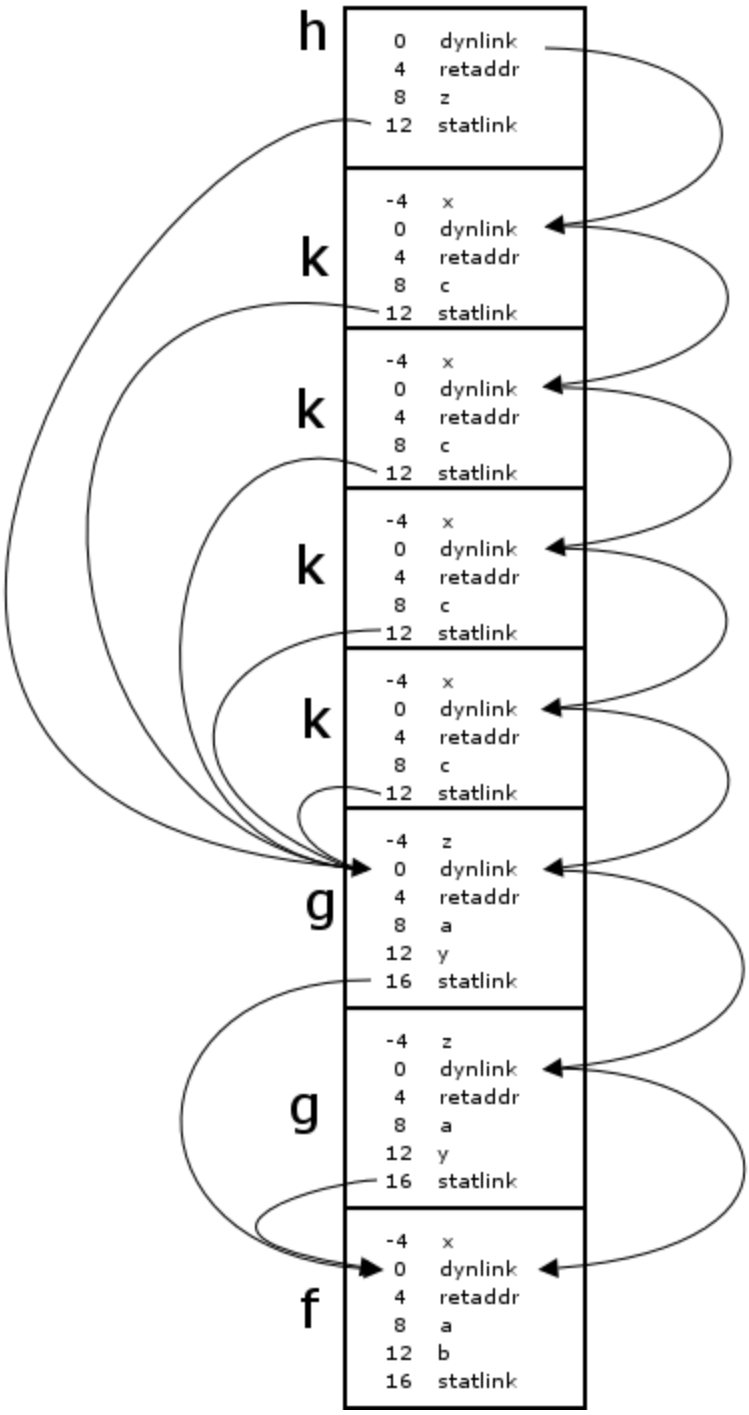
    g(0, 0);
}
```

The call sequence goes f → g → g → k → k → k → k → h. What gets printed? It depends on whether we have static or dynamic scoping.

If dynamically scoped, use a-lists or central reference tables.

If statically scoped, use *static links* (or *displays*). In this case we rely on the compiler to generate code that says:

> "To find x from the body of k, take two steps down the static chain and look in offset -4".



Heap Storage

TODO

Garbaage Collection

TODO

Process Management

TODO

Thread Management

TODO

Summary

We've covered:

- ✅ -
- ✅ -